

Shift

多项式平移|连续点值平移

多项式平移

多项式平移是简单情况的多项式复合变换，给出 $P(x)$ 的系数和一个常数 c ，求 $P(x+c)$ 的系数，即。

分治法

令

那么

$P(x)$ 的系数为二项式系数，那么

其中 n 为多项式乘法的时间。

Taylor 公式法

对 $P(x)$ 在 $-c$ 处应用 Taylor 公式，有

那么

观察到对于 $x = -c$ 有

令

那么

二项式定理法

考虑二项式定理 那么

得到的结果与上述方法相同。

连续点值平移

▼ 例题 [LOJ 166 拉格朗日插值 2](#)

给出度数小于等于 n 的多项式 $f(x)$ 的连续点值 $f(x_i)$ ，在模 p 意义下计算，其中 p 为质数。

Lagrange 插值公式法

考虑 [Lagrange 插值公式](#)

上式虽然是卷积形式但不能保证分母上 $f(x_i) \neq 0$ ，所以下面仅考虑 $f(x_i) \neq 0$ 的情况，其他情况（如系数在模素数意义下时须避免 $f(x_i) = 0$ 系数的分母出现零）可以分类讨论解决，令

那么对于 x 有

实现中取 n 需要的部分截断可求出更多点值，且可利用循环卷积。

对问题稍加修改，假设对于某个 x_i 给出的点值为 y_i ，我们可以计算 y_i ，视作平移 x_i 的点值为 y_i 。

Lagrange 插值公式也给出了通过维护一些前后缀积的线性计算单个点值的方法。

应用

同一行第一类无符号 Stirling 数

▼ 例题 [P5408 第一类斯特林数·行](#)

在模素数 p 意义下求，其中 p 为质数。

考虑

其中 x 为上升阶乘幂，令 $y = x$ 那么

通过多项式平移可在 $O(n)$ 求出，问题被缩小为原先的一半即求出 y 的系数，那么

模素数意义下阶乘

▼ 例题 [P5282【模板】快速阶乘算法](#)

求 $n!$ ，其中 p 为素数且 $p > n$ 。

令 $x = n$ 和 $y = n$ 那么

其中 x 可在 $O(\sqrt{n})$ 时间计算，我们希望可以快速计算上式的前半部分。

多项式多点求值

系数的计算可用上述多项式平移算法在 $O(n)$ 时间得到，但多点求值计算 需要 $O(n^2)$ 时间。

连续点值平移

令 $f(x)$ ，我们可以用 n 个点值 唯一确定这个次数为 $n-1$ 的多项式，又

所以只需 n 个点值可以唯一确定 $f(x)$ ，那么使用连续点值平移计算 $f(x_1)$ （即平移 x_1 的点值 为 x_1 ）和 $f(x_2)$ （即平移 x_2 的点值 为 x_2 ）后将这两者的对应点值相乘即得 $f(x_1 x_2)$ 。

由 $f(x)$ 计算 $f(x_1)$ 考虑

额外增加的一个点值使用线性时间的算法即可。那么在开始时维护 n 个点值 后使用连续点值平移来倍增地维护这些点值，有

而我们只需要约 $\log n$ 个点值，所以时间复杂度为 $O(n \log n)$ 。

模素数意义下二项式系数前缀和

▼ 例题 [LOJ 6386 组合数前缀和](#)

求 $\sum_{i=0}^n \binom{n}{i} x^i$ ，其中 x 。

考虑使用矩阵描述 这一步递推，我们有

类似的可以将二项式系数前缀和的递推描述为

注意矩阵乘法的顺序，那么

令 M ，考虑维护矩阵

的点值 即 $x^0, x^1, \dots, x^{n-1}$ 和 1 ，又

且矩阵右下角元素恰为我们在阶乘算法中所维护的，那么

可在 $O(n \log n)$ 时间完成计算。

模素数意义下调和数

▼ 例题 [P5702 调和级数求和](#)

求 $\sum_{i=1}^n \frac{1}{i}$ ，其中 p 为素数且 $p \nmid n$ 。

记，一步递推为

那么

在这里 B_n 和 A_n 为第一类无符号 Stirling 数。维护点值矩阵的方法同上。

整式递推

对于更一般的情况，类似于上述快速阶乘算法的案例，我们期望得到一个怎么样的算法？

▼ 例题 [P6115【模板】整式递推](#)

现有数列 f_i 满足，其中 f_i 为不超过 n 次的多项式。

给定所有 f_i 的系数，和 n ，求 f_n 。对 998244353 取模。 $n \leq 10^5$ ，时限 1s。

为了更系统地描述上述几道例题中构造矩阵的过程，我们引入 [矩阵](#) 的概念。

为了实现整式递推，我们应当注意到快速阶乘算法过程中，我们维护的点值其实并不是 f_i ，而是 $f_i(x)$ ，即 f_i 一对点值之间的倍数关系。

由于整式递推阶数 n 不止是 n 了，我们就不能直接维护一对数之间的倍数关系了；而是维护出一对 n 维向量之间的线性变换，即 $f_i(x)$ 的一个矩阵，矩阵的每一项对应于某个多项式的一个点值。

容易发现，对于一般的整式递推远处系数求值问题，我们可以构造

设

我们先撇开前面的 f_i 因子不论，我们现在要维护 $f_i(x)$ 这种形式的量，其中乘法自右往左。

容易发现 $f_i(x)$ 是一个各项次数不高于 n 的 n 矩阵，只用 n 个值即可维护。

于是我们维护出 $f_0, f_1, f_2, \dots, f_{n-1}$ 这几个 n 矩阵的点值，然后用类似于快速阶乘算法的方式暴力进行多项式点值平移和倍增就好了。

具体地，为了让 f_i 抬高，我们这么干：

1. 在 $O(n)$ 时间内获取 $f_0, f_1, f_2, \dots, f_{n-1}$ 。
2. 在 $O(n)$ 时间内获取 $f_1, f_2, f_3, \dots, f_n$ 。
3. 在 $O(n)$ 时间内获取 $f_2, f_3, f_4, \dots, f_{n+1}$ 。
4. 计算 f_n 。

我们每轮花费 $O(n^2)$ 的复杂度进行平移；同时，我们每轮只用做 $O(n)$ 次矩阵乘法，复杂度可以认为是 $O(n^2)$ 。

最后，我们只用做到 f_n 即可。

之前的 f_i 因子可以用类似的方法解决。

这样，我们预处理的复杂度即为 $O(n^2)$ 。

考虑查询，我们只用 $O(n)$ 次向量与矩阵的乘法，以及 $O(n^2)$ 次暴力转移。

容易发现这部分计算并不是复杂度瓶颈。

因此，该算法的总复杂度为 $O(n^3)$ 。

编码时，我们可以使用循环卷积的技巧来减小 NTT 的常数。

在实际应用时，我们往往是对一个已知的微分有限的 GF 提取其远处系数，从而 $O(1)$ 均为常数，也即做到了 $O(1)$ 的远处系数求值。

参考文献

- Alin Bostan, Pierrick Gaudry, and Eric Schost. Linear recurrences with polynomial coefficients and application to integer factorization and Cartier–Manin operator.
- Min_25 的博客
- [ZZQ 的博客 - 阶乘模大质数](#)

本页面最近更新：2024/10/30 21:50:12, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [hly1204](#), [c-forrest](#), [Enter-tainer](#), [myeeye](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用